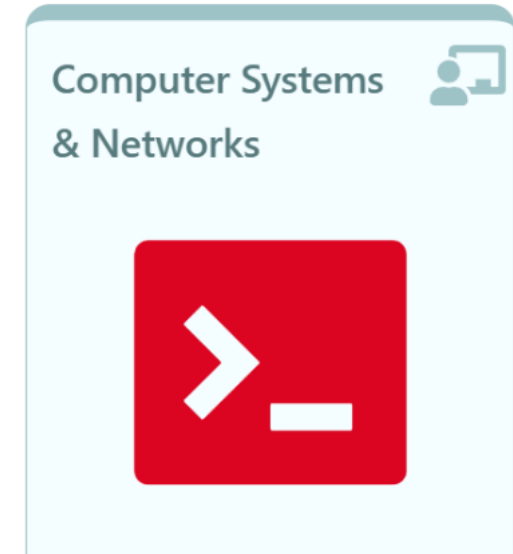
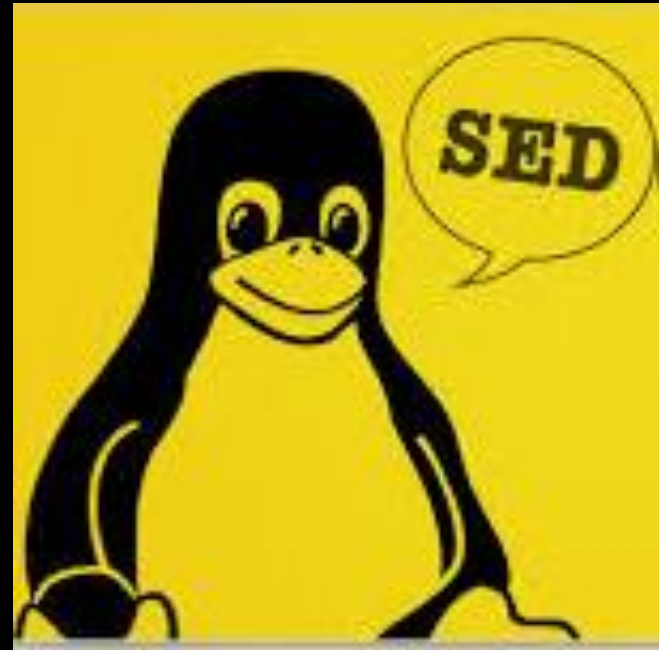


Computer Systems & Networks



sed

for finding, replacing,
deleting, and inserting text.



sed

- stream editor

sed is a special editor for **modifying files** automatically

- make global changes to a large file
- Allows a script make changes to a file

sed requires **an input file** and a **command** that lets sed know what actions to apply to the file

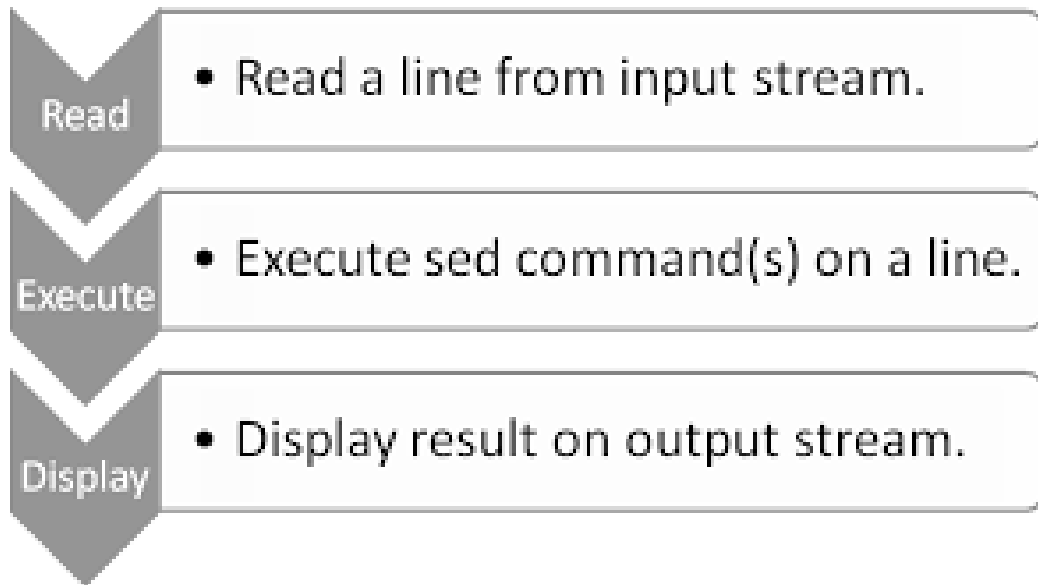
sed uses:

- Text substitution
- Selective printing of text files
- Replace text in-place (modify the file)
- Non-interactive editing of text files etc.

⚠ *irreversible unless backed up!!*

sed

- stream editor



sed can be used:

- at the command-line, or
- within a shell script to edit a file non-interactively

= *no need to open a text editor*
(ie I could replace all occurrences of 2020 with 2030)

Syntax

\$sed '/pattern/ command' my_file

Where ***pattern*** is a regex and
command can be one of:

s = search & replace, or

p = print, or

d = delete, or

i=insert, or

a=append etc.

Substitute Text (s)

\$ sed 's/cat/dog/'

The diagram shows the command 's/cat/dog/' with labels: 's' is the command, '/' is the delimiter, 'cat' is the pattern, and 'dog' is the replacement.

\$sed 's/WIT/SETU/' staff_file

- i. echo every line from `staff_file` to STDOUT,
- ii. change *the first occurrence* of the string 'WIT' on each line into 'SETU'

NOTE: `sed` is line-oriented

- replaces the first occurrence & moves to next line
- if you wish to change *every occurrence* on each line, make it a 'greedy' global search:

\$sed 's/WIT/SETU/g' staff_file

Remove (delete) Lines (**d**)

\$sed '/WIT/d' file.txt

echo every line from `file.txt` to STDOUT,
deleting lines matching the pattern.

\$sed '/^\$/d' file.txt #what is this action?

Replace first occurrence of WIT only on line 2:

\$sed '2s/WIT/SETU/' file.txt

Replace all occurrences on line 2:

```
$sed '2s/WIT/SETU/g' file.txt
```

Replace first occurrences on every line:

```
$sed 's/WIT/SETU/' file.txt
```

Replace all occurrences on every line:

```
$sed 's/WIT/SETU/g' file.txt
```


Print only lines that matching pattern (**p**)

\$sed -n '/SETU/p' file.txt

-n:	suppress normal output (by default, sed prints every line)
/pattern/	matches lines containing the given pattern.
p	print only matching lines

Case-Insensitive Matching (**I**)

\$sed -n '/SETU/Ip' file.txt

I	case-insensitive, so it will match SETU, setu, Setu, etc.
----------	---

Escape Symbol

\$ sed 's/cat/dog/'

delimiter
pattern
command replacement

Replace **/bin** with **/usr/local/bin** in a file called **my_script**, and write the result to a new file called **new_script**:

```
sed -e 's/\bin/\usr/local/bin/' my_script > new_script
```



AHH, it's so confusing looking!

Tip: You could also use a different delimiter to avoid escaping slashes, e.g., **#**, **@**, **|**, **:**, etc., as long as you're **consistent**

```
$sed 's#/bin#/usr/local/bin#' my_script > new_script
```

Wildcards

You want to take every line that starts with a number in your file and
<<perform some action>>

TIPS:

`sed -E` → enables **extended regex** = can use + without escaping.

`^ ([0-9]*)` → matches zero or more digits at the start of the line *or*

`^ ([0-9]+)` → matches one or more digits at the start of the line

*Experiment with using some echo statements in your output to test the outputs;
you may wish to add in some lines into **employee.txt** that do and do not start with
numbers*

Redirecting output

- By default the output is written to STDOUT.

```
sed -e 's/input/output/' my_file > new_file
```

-e

is mainly for specifying multiple commands on the same sed invocation
is optional when running **only one** command

```
sed -i -e 's/input/output/' my_file
```

-i

edits my_file directly, replacing all occurrences of “input” with “output”,
instead of printing the modified content to stdout.

Examples to try out

- Print out only directories (i.e. they begin with a “d” in `ls -l`)

```
$ls -l | sed -n -e '/^d/ p'
```

- You wanted to delete all lines that start with the comment symbol '#' you could use

```
$sed -e '/^#/ d' my_file
```

Delete Lines down to the end (**d**)

You can also use the special line number '\$' to mean 'end of file'.

- if you wanted to delete the **range of lines**, from line 11 to the **last line**

```
$sed -e '11,$ d' my_file
```

Print a Range of Lines (**p**)

You can also use a **pattern-range** form i.e. match **all lines from the first line that matches *caroline* up to and including the first line that matches *rosanne*:**

```
$sed -n -e '/caroline/,/rosanne/p' employee.txt
```

SUMMARY: *SOME* reasons why we use sed

Simple find & replace (global)	sed 's/WIT/SETU/g' myfile Fixing typos across files
In-place edit	sed -i.bak 's/WIT/SETU/g' file .bak saves your life more often than you'd think
Replace only on matching lines	sed '/ERROR/s/foo/bar/' log.txt Only substitute on lines that contain ERROR
Delete lines by number	sed '1,10d' file #Removing headers sed '\$d' # remove last line sed '11,\$d' # keep only first 10 lines
Delete lines by pattern	sed '/^#/d' myfile #Removing comments sed '/^\$/d' # remove empty lines
Print only matching lines (grep-like)	sed -n '/pattern/p' file
Print a range between two patterns	sed -n '/start/,/end/p' file
Wrap or reuse matched text with &	sed 's/[0-9]\+/(&)/g' file #Find every number in each line and wrap it in ()
Change delimiters to avoid using \ escape	sed 's#/usr/bin#/usr/local/bin#g' file #for paths, URLs etc.

[T](#) > [Home](#) > [Computer Systems](#) > [03: Loops, Menus](#) > [Week 3 Class 1](#) > [Lab](#)

Advancing Scripting Skills

01. Conditionals

02. Loops

- ### 03. Menu driven scripts

04. sed

05. EXER Add new records

06. Cisco NetAcad

```
$ sed -n '/SUSE/lp' distros.txt #except the ones matched by X
```

Make a copy of the **distros.txt** file, named **tempdistros.txt** and practice deleting sections of the file

```
$ sed '1d' tempdistros.txt #deletes 1st line
```


Try out this weeks Lab

T > Home > Computer Syste... > 03: Loops, Men... > Week 3 Class 1... > Lab

Advancing Scripting Skills

01. Conditionals

02. Loops

03. Menu driven scripts

04. sed

05. EXER Add new records

06. Cisco NetAcad

05. EXER Add new records

EXERCISE

Write a script that can be used to add a new record (employees details) to your **employee.txt** file

TIPS

Think about this logically:

- 1st check if the file exists and maybe do you want to create it if it's not there? Do you want to use to receive error messages Customised feedback?
- Then you want to give appropriate variable names in order to the positional parameter, the 1st parameter being the employee's ID, the second their name, the third their position in the company. 4th the dept they belong to and finally, 5th the expenses they claimed.
- Whats next? Time to take the details in and append them to your **employee.txt** file

ERROR CHECKING:

Error checking is an **absolute MUST** especially when you're taking user input into a system to ensure that the expected input is in the expected format i.e. email addresses, phone numbers, postcodes etc. are all in the expected format

Advancing Scripting Skills

01. Conditionals

02. Loops

03. Menu driven scripts

04. sed

05. EXER Add new records

06. Cisco NetAcad

06. Cisco NetAcad

CA · Chapter Exams

For the purpose of this module, we will today complete the final two chapters and their associated Chapter Exams.

- For the 5% CA, you are required to have completed the chapter exams from Chapter 1 up to and including Chapter 11



5% Cisco
NetAcad
chapter exams

Final NetAcad 5% CA

Completed Chapter
Exams *up to and
including* Chapter
11

